Distributed Campus Lost & Found

Design, Implementation and Evaluation of a Two-Node Distributed Prototype

University of Bologna — Distributed Systems | Student: [insert official name]

Report draft — 6 November 2026

Abstract

This project implements a small distributed lost-and-found platform for two simulated university campuses: Cesena and Forlì. Each campus runs an independent Python Flask service, exposes a browser interface and REST endpoints, and stores item reports in its own SQLite database. The application supports local operations while communication between campuses is unavailable. When connectivity returns, either node can initiate an HTTP/JSON synchronization exchange. Instead of overwriting changes using local timestamps, the system maintains immutable report versions and vector clocks, detects concurrent updates and presents the conflicting candidates for explicit human resolution. The prototype illustrates distributed architectures, availability during partitions, replication, causal ordering and the consistency trade-offs covered in the Distributed Systems course.

Keywords: distributed systems; REST; replication; vector clocks; CAP theorem; eventual consistency; SQLite; Flask.

1. Motivation and Objectives

University students may report a lost or found object from different campus locations. A centralized application could provide a common catalog, but the aim of this project is not merely to build a website. It is to study what happens when different locations can operate independently, temporarily lose communication and then reconcile their records. The scenario is deliberately simplified to make the behavior observable and testable.

The approved project proposal defined the following goals:

1. Understand the structural components of a distributed application.
2. Investigate the availability of local operations during a communication failure.
3. Exchange changes between independently stored datasets.
4. Detect and resolve concurrent modifications without silently discarding information.

The implementation covers two simulated campuses and is intended for one student's educational work. It does not aim to provide a production campus service.

2. Requirements and Scope

2.1 Functional requirements

- A user can create a **LOST** or **FOUND** report, specifying its title, category and optional description.
- Each campus can list and search reports, and change a report's editable attributes or status.
- Each campus persists its own report history without requiring a connection to the other campus.
- Users can manually request two-way synchronization.
- A network-partition simulation can block inter-node replication without disabling local reads and writes.
- If a report is updated independently at both campuses, the system identifies a **version conflict** and retains the concurrent values.
- A user can explicitly resolve a conflict by selecting one current candidate. The resolution is subsequently replicated.
- The UI can suggest potentially related lost and found reports using a basic keyword heuristic. Such suggestions are distinct from version conflicts.

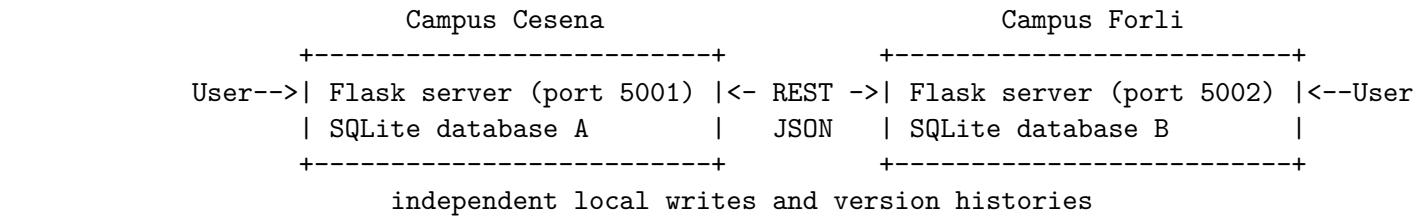
2.2 Non-functional requirements

The solution should run on one student laptop without external cloud infrastructure, use understandable technologies, preserve records in a local database, make the distributed effects repeatable in a demonstration, and have automated tests for the key algorithms. Security hardening, global scalability and automated failure recovery are outside this prototype's scope.

3. Architecture and Technologies

3.1 Logical structure

Two independent **Flask + SQLite** instances run on separate localhost ports:



Each node can accept user requests without consulting the peer. There is **no shared SQL database** and no centralized coordinator. In the test environment both processes run on one physical computer; they represent independent logical nodes rather than two physical machines.

The backend separates application logic (`campus/core.py`) from HTTP endpoints (`campus/server.py`). Each node's identity, SQLite path, port and peer URL can be configured. The front end uses HTML, CSS and JavaScript; JSON is used for HTTP payload serialization.

3.2 REST communication

The public-facing API includes `POST /api/items`, `GET /api/items`, `GET /api/items/<id>` and `PATCH /api/items/<id>`. For distributed behavior, `POST /api/sync` initiates a synchronization

request to the other node's `POST /api/replicate` endpoint. The peer validates incoming versions, inserts those it does not already store, and responds with its own version history. The initiator then merges any versions that it was missing.

The API is intentionally simple, allowing students to inspect calls through a browser or command-line HTTP client. HTTP failures produce visible error responses; local changes remain stored.

4. Versioning, Replication and Consistency

4.1 Immutable report histories

Each report receives a globally unique UUID. Each creation, modification or conflict resolution produces a new, immutable version with its own UUID, a complete data snapshot, the originating node and a **vector clock**. The SQLite `versions` table records these snapshots and a unique index on version ID prevents the same version being inserted repeatedly. Older versions are retained for traceability.

4.2 Causality with vector clocks

For version vectors V and W, V **dominates** W when every component of V is greater than or equal to the corresponding component of W and at least one component is strictly greater. If neither dominates the other, then the two versions are concurrent in the causal history.

For example, assume Cesena created a report and replicated it to Forlì, giving both nodes the version vector `{cesena: 1}`. During an outage:

- Cesena edits that report, producing `{cesena: 2}`.
- Forlì independently edits the same report, producing `{cesena: 1, forli: 1}`.

Neither vector dominates the other. The system therefore keeps **two current candidates**, marks the report as a conflict and prevents ordinary modifications until the user decides how to resolve it. Local timestamps cannot reliably solve this problem because independent physical clocks do not establish causality.

4.3 Resolution and convergence

An explicit resolution selects a candidate's data, computes the component-wise maximum of all conflicting vectors, and increments the resolving node's component. A resolution performed by Cesena in the example obtains `{cesena: 3, forli: 1}`. This newly created vector dominates the two previous heads. The previously conflicting versions remain stored as immutable history.

Synchronization exchanges complete version histories rather than just the latest report values. The operation is idempotent for a given version UUID. Once the network is available, the set of writes stops changing, synchronization occurs, and detected conflicts have been resolved, both nodes converge to the same current report representation. **Synchronization is manually triggered** and is not a background service.

4.4 CAP theorem and network failures

This prototype illustrates one of the trade-offs described by the CAP theorem: during a communication partition, the nodes continue accepting **local** writes, sacrificing an immediately consistent

cross-campus view. The application thus favors write availability in this particular failure scenario. It does not guarantee strict global consistency while communication is impossible, and it should not be labeled a universally high-availability or partition-proof product.

The partition simulation disables incoming and outgoing replication requests on both nodes. It does **not** cut the computer's real network connection or stop Flask; this makes the experiment reproducible. The local SQLite databases retain their own data, while a synchronization attempt during the simulated outage fails visibly with HTTP 503.

5. Lost/Found Suggestions Versus Version Conflicts

The prototype distinguishes two ideas that could otherwise be confused:

Potential item match: A Cesena LOST report and a Forlì FOUND report have *different* UUIDs. A small case-insensitive title-keyword overlap algorithm proposes a possible relationship between them, provided their categories are compatible. It cannot prove the objects are identical and never merges their data automatically.

Concurrent version conflict: Two independent edits concern the *same report UUID*. The vector clocks are incomparable, indicating concurrent histories. The application retains both candidate snapshots and requires a deliberate resolution.

This distinction prevents a plausible description match from being misinterpreted as a database consistency failure.

6. Implementation and Reproducible Evaluation

6.1 Environment and execution

The project uses Python, Flask, SQLite, HTML, CSS and JavaScript. Instructions are included in `README.md`. Two terminal processes start the Cesena and Forlì services; their independent SQLite files are created automatically. The script `python -m scripts.demo` exercises the main distribution scenario against two running servers.

6.2 Demonstration protocol

Step	Action	Expected observation
1	Create a report at Cesena	Only Cesena has the report before synchronization
2	Synchronize Cesena with Forlì	Both databases now have the report
3	Disable replication at both nodes	Synchronization requests fail, but local actions still succeed
4	Edit the same report independently	Two causally divergent versions are created
5	Restore the link and synchronize	Both nodes show a conflict with two candidates
6	Choose one candidate as resolution	A causally later merged version is recorded

Step	Action	Expected observation
7	Synchronize again	Both nodes converge to the same current report
8	Create a separate FOUND report	Keyword matching may suggest a relationship, not a version conflict

6.3 Automated verification and observed results

The project contains dependency-free core tests in `tests/test_core.py` and additional Flask route tests in `tests/test_flask_optional.py`. In the preparation environment, **7 core tests completed successfully**. These tests verified vector clock comparisons, local CRUD/validation, persistence, real HTTP/JSON data exchange through a test peer server, two-way idempotent replication, an outage, concurrent conflict detection, manual resolution, convergence and possible matching. **Two Flask-specific endpoint tests were skipped** because the current preparation environment did not contain the Flask package. The Flask web server and full `scripts.demo` run therefore remain to be executed on a machine where Flask can be installed. No browser-level validation or unobserved runtime result is claimed in this draft.

A complete acceptance run should execute `python -m unittest discover -s tests -v` *after installing Flask*, then start both web servers and run `python -m scripts.demo`. The output should be recorded for the final submitted report once verified.

7. Limitations and Future Work

This prototype consciously trades scale and security for clarity of the distribution mechanisms:

- **Two nodes only.** The topology is fixed and statically configured; membership discovery is not implemented.
- **Manual, full-history sync.** Every request exchanges all known versions, making the protocol inefficient for large datasets. A production version would use incremental changes, cursors or compacted logs, plus retries and backoff.
- **No access control or peer authentication.** HTTP is restricted to localhost, and administrative simulation and resolution endpoints are not protected. The system is unsuitable for public deployment or sensitive personal information.
- **No global transaction.** Two nodes can legitimately show different report states during an outage.
- **Basic keyword matching.** Matching is an optional suggestion with possible false positives and negatives, not a machine-learning or identity-verification system.
- **No background fault detection.** Synchronization is initiated manually; no automatic peer-health monitor or automatic reconnection protocol exists.

Possible extensions include authenticated communication, HTTPS, automatic retry/sync scheduling, incremental replication, a third node, user accounts, stronger matching and controlled conflict-resolution policies.

8. Conclusions

The project demonstrates how two separate campus applications can remain locally usable despite a simulated network partition, exchange history after reconnection and expose concurrency instead of silently losing user edits. An append-only version store and vector clocks connect the implementation with the course topics of distributed architectures, REST communication, replication and consistency, fault tolerance and the CAP trade-off. Within its stated educational scope, the system provides a compact environment for discussing both the practical benefits and the inherent limits of distributed computation.

References and Supporting Material

1. University of Bologna, *Distributed Systems*, course materials and project rules supplied by the course staff (consulted for project context).
2. Source code and instructions: https://github.com/YuetingZhao/distributed-campus-lost-found (**target repository; local deliverables must still be uploaded**).
3. Project source: `campus/core.py`, `campus/server.py`; automated tests: `tests/`.

Submission note: This is an editable draft, not a claim that the required final PDF has been validated by the instructor. Update the student name, confirm all Flask-level tests, check the course's official report template, and submit through the required forum procedure.